Prepared by: *Mariya Danilova*

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues Found
- Findings
  - High
    - [H-1] Stub verifier accepts any proof — full prize pool drainable by any user
    - [H-2] Double-spend check uses wrong mapping key — same treasure claimable 10 times
    - [H-3] Treasure secrets are trivially brute-forceable — hunt solvable without playing
    - [H-4] Duplicate entry in `ALLOWED_TREASURE_HASHES` — only 9 unique treasures exist
    - [H-5] Negative field elements in `Prover.toml.example` — 6 of 10 treasures permanently unclaimable
  - Medium
    - [M-1] CEI pattern violated in `emergencyWithdraw` and `withdraw`
  - Low
    - [L-1] `updateVerifier` sets verifier without zero-address check
    - [L-2] `Claimed` event emits `msg.sender` instead of `recipient`
    - [L-3] `nonReentrant` modifier applied to only one of three ETH-sending functions
    - [L-4] `recipient` public input unconstrained in ZK circuit — binding relies solely on proof system
    - [L-5] ETH transfer to smart contract recipient may permanently brick a treasure claim
    - [L-6] `withdraw()` missing access control — any address can trigger owner withdrawal
  - Informational
    - [I-1] 11 custom errors declared but never used
    - [I-2] Mixed error-handling styles (`require` strings vs. custom errors)
    - [I-3] Unspecific Solidity pragma
    - [I-4] PUSH0 opcode — EVM version compatibility
    - [I-5] Unused state variable `_treasureHash`
    - [I-6] `bb` (Barretenberg) missing from devcontainer — build pipeline incomplete

Protocol Summary

TreasureHunt is an on-chain treasure hunt game where players solve off-chain puzzles and submit Barretenberg/Noir ZK proofs to claim ETH rewards. The contract holds up to 100 ETH (10 treasures × 10

ETH each), and a valid ZK proof binding a `treasureHash` to a `recipient` address is required to claim each reward.

Competition: CodeHawks 2026-04-snarkeling

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time-boxed engagement and the findings represent best-effort analysis. The report does not constitute investment advice and should not be relied upon for deployment decisions without further review and remediation.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix.

Audit Details

Commit Hash: `aed232c` (initial commit)

Scope

```
contracts/src/
├── TreasureHunt.sol        (165 nSLOC)
└── Verifier.sol            (11 nSLOC)
circuits/src/
├── main.nr                 (ZK circuit)
└── tests.nr                (circuit tests)
Total nSLOC: 176 (Solidity) + Noir circuits
```

Roles

Role	Privileges
Owner	Fund, pause/unpause, update verifier, emergency withdraw, withdraw after hunt ends
Player	Call `claim()` with a valid ZK proof to receive 10 ETH

Executive Summary

Two critical vulnerabilities were found that, in combination, allow any user to drain the entire 100 ETH prize pool without a valid ZK proof and without any per-treasure uniqueness restriction. The contract is not safe to deploy with funds in its current state.

Static analysis (Slither, Aderyn) confirmed the critical uninitialized-variable finding and surfaced additional low/informational issues.

Issues Found

Severity	Count
High	5
Medium	1
Low	6
Informational	6
Total	**18**

Findings

High

[H-1] Stub verifier accepts any proof — full prize pool drainable by any user {#h-1-stub-verifier-accepts-any-proof}

Severity: High

Description:

`BaseZKHonkVerifier.verify()` in `Verifier.sol` unconditionally returns `true` without performing any cryptographic verification:

```
// Verifier.sol:11-13
function verify(bytes calldata, bytes32[] calldata) external pure returns
(bool) {
    return true;
}
```

`TreasureHunt.claim()` delegates proof validation entirely to this function. Since it always returns `true`, the ZK-proof guard is permanently neutralised and any caller passes verification with arbitrary bytes.

Impact:

Any non-owner address can call `claim()` with garbage proof bytes and receive 10 ETH. By repeating this 10 times, an attacker drains all 100 ETH, locking out every legitimate hunter.

Proof of Concept:

```
// Attacker calls with empty proof and any non-zero hash
hunt.claim(bytes(""), keccak256("fake"), payable(attackerWallet));
// verifier.verify() returns true → 10 ETH transferred
// Repeat 10 times → 100 ETH drained
```

Recommended Mitigation:

Replace the stub with the Barretenberg-generated Honk verifier produced by `build.sh`. Gate deployment scripts on the real verifier being present. Consider adding a compilation check or constructor assertion that `verifier.verify(bytes(""), new bytes32[](0)) == false`.

[H-2] Double-spend check uses wrong mapping key — same treasure claimable 10 times {#h-2-double-spend-check-uses-wrong-mapping-key}

Severity: High

Description:

The double-spend guard on line 88 reads from `claimed[_treasureHash]` (an immutable state variable that is declared but **never assigned** in the constructor, defaulting to `bytes32(0)`):

```
// TreasureHunt.sol:35 — never initialized
bytes32 private immutable _treasureHash;

// TreasureHunt.sol:88 — checks wrong key (always bytes32(0))
if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);

// TreasureHunt.sol:104 — writes correct key (function parameter)
_markClaimed(treasureHash);
```

The check and the write permanently use different mapping keys. For any `treasureHash != bytes32(0)`, the `AlreadyClaimed` guard **never fires**. The same treasure hash can be claimed repeatedly until `claimsCount` reaches `MAX_TREASURES`.

This is confirmed by the test suite: `testClaimDoubleSpendReverts` has `vm.expectRevert()` commented out, meaning the test asserts that double-spending *succeeds*.

Impact:

A single attacker with one valid proof can replay `claim()` with the same `treasureHash` up to 10 times, draining all 100 ETH and blocking every other participant permanently.

Recommended Mitigation:

Change line 88 from:

```
if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);
```

to:

```
if (claimed[treasureHash]) revert AlreadyClaimed(treasureHash);
```

Remove the unused `bytes32 private immutable _treasureHash` declaration on line 35. Uncomment `vm.expectRevert()` in the double-spend test and verify it passes.

[H-3] Treasure secrets are trivially brute-forceable — hunt solvable without playing {#h-3-trivially-brute-forceable-secrets}

Severity: High **File:** `circuits/src/main.nr`, `circuits/src/tests.nr`

Description:

The `ALLOWED_TREASURE_HASHES` array is baked into the public circuit bytecode. The test file directly reveals that the production secrets are small integers:

```
// tests.nr:30
let treasures: [Field; 10] = [1, 2, 3, 4, 5, 6, 7, 8, 10, 10];
```

The first allowed hash (`1505662313...502`) is confirmed by the test to equal `pedersen_hash([1])`. Since Pedersen hash parameters are public, anyone can compute `pedersen_hash(n)` for `n = 1..100` offline in seconds and match every entry in `ALLOWED_TREASURE_HASHES`, recovering all secrets before the hunt opens.

Impact:

Every treasure secret is recoverable from public data without solving any off-chain puzzle. An attacker can pre-compute all valid proofs and drain the prize pool at contract deployment.

Recommended Mitigation:

Use cryptographically random 256-bit Field elements as secrets, generated off-chain and never committed to test fixtures in the same repository. Do not derive secrets from sequential integers.

[H-4] Duplicate entry in `ALLOWED_TREASURE_HASHES` — only 9 unique treasures exist {#h-4-duplicate-treasure-hash}

Severity: High **File:** `circuits/src/main.nr:64-65`

Description:

Indices 8 and 9 of `ALLOWED_TREASURE_HASHES` are identical:

```
-96143505731729358009482648278657287353323570118332983112409184763554787 10
92,
-96143505731729358009482648278657287353323570118332983112409184763554787 10
92  // ← exact duplicate
```

The test confirms secret `9` is missing and secret `10` fills both slots (`treasures = [1,2,3,4,5,6,7,8,10,10]`). Only 9 unique treasure secrets exist despite the contract expecting 10.

Impact:

- One treasure slot can never be claimed by a unique solver — the puzzle for it does not exist.
- Combined with the broken double-spend check (H-2), whoever holds secret=10 can claim both slots, earning 20 ETH while depriving a legitimate hunter.
- `withdraw()` (owner recovery after all 10 claims) may never trigger correctly if one slot is unclaimed.

Recommended Mitigation:

Generate 10 distinct secrets and verify `ALLOWED_TREASURE_HASHES` has no duplicate entries before deployment. Add a constructor assertion or deployment script check enforcing uniqueness.

[H-5] Negative field elements in `Prover.toml.example` — 6 of 10 treasures permanently unclaimable {#h-5-negative-field-elements}

Severity: High **File:** `circuits/Prover.toml.example`

Description:

`Prover.toml.example` stores 6 of the 10 `treasure_hash` values as **negative decimal** integers (e.g. `-5602859741022561807370900516277986970516538128871954257532197637239594541050`). Nargo requires field elements to be represented as **non-negative** integers. Attempting to generate a witness for any of these indices fails immediately:

```
Failed to deserialize inputs: The value passed for parameter
`treasure_hash` is invalid:
Expected witness values to be integers, but
`-5602859741022561807370900516277986970516538128871954257532197637239594541050`
failed with `invalid digit found in string`
```

Affected indices (confirmed by running `build.sh`):

Index	Treasure secret	Hash sign	Claimable?

Index	Treasure secret	Hash sign	Claimable?
0	1	positive	✓
1	2	**negative**	×
2	3	**negative**	×
3	4	positive	✓
4	5	positive	✓
5	6	**negative**	×
6	7	**negative**	×
7	8	positive	✓
8	9	**negative**	×
9	10	**negative**	×

Impact:

6 treasures can never produce a valid proof. No valid proof means no valid `proof.bin` fixture, no valid `Verifier.sol`, and no on-chain claim — 60 ETH is locked in the contract permanently with no recovery path except `emergencyWithdraw`.

Root Cause:

Noir's Pedersen hash returns field elements in the BN254 scalar field. When the result exceeds `p/2`, it is sometimes represented as a negative number (`result - p`). The TOML parser requires the canonical positive representation.

Recommended Mitigation:

Convert all negative hashes to their positive modular equivalents before writing to `Prover.toml.example`. For each negative value:

```
positive = field_prime + negative_value
# BN254 field prime p:
#
21888242871839275222246405745257275088548364400416034343698204186575808495
617
```

Example for index 2 (`treasure = 3`):

```
21888242871839275222246405745257275088548364400416034343698204186575808495
617
+
(-56028597410225618073709005162779869705165381288719542575321976372395945
4
1050)
```

```
=
16285383130816713414875505228979288118031826271544080086166006549336213954
567
```

Add a verification step to `build.sh` that asserts all hashes in `Prover.toml.example` are non-negative before proceeding.

Medium

[M-1] CEI pattern violated in `emergencyWithdraw` and `withdraw` {#m-1-cei-pattern-violated}

Severity: Medium

Description:

Both `emergencyWithdraw()` and `withdraw()` emit events **after** the external `.call`, violating the Checks-Effects-Interactions pattern:

```
// TreasureHunt.sol:279-282
(bool sent, ) = recipient.call{value: amount}("");
require(sent, "ETH_TRANSFER_FAILED");
emit EmergencyWithdraw(recipient, amount); // ← after external call
```

While `emergencyWithdraw` is guarded by `paused` and `withdraw` sends only to the immutable `owner`, off-chain monitoring tools that rely on event ordering may receive incorrect state signals if the call fails mid-execution.

Recommended Mitigation:

Move event emissions before the external `.call` in both functions:

```
emit EmergencyWithdraw(recipient, amount); // emit first
(bool sent, ) = recipient.call{value: amount}("");
require(sent, "ETH_TRANSFER_FAILED");
```

Low

[L-1] `updateVerifier` sets verifier without zero-address check {#l-1-updateverifier-no-zero-address-check}

Severity: Low

Description:

The constructor validates `_verifier != address(0)`, but `updateVerifier()` does not:

```
// TreasureHunt.sol:263-269
function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
    verifier = newVerifier; // no address(0) check
    emit VerifierUpdated(address(newVerifier));
}
```

An accidental `updateVerifier(address(0))` would brick the contract — all subsequent `claim()` calls would revert on the verifier call.

Recommended Mitigation:

```
if (address(newVerifier) == address(0)) revert InvalidVerifier();
```

[L-2] `Claimed` event emits `msg.sender` instead of `recipient` {#l-2-claimed-event-wrong-address}

Severity: Low

Description:

```
// TreasureHunt.sol:111
emit Claimed(treasureHash, msg.sender); // should be recipient
```

The ETH is sent to `recipient`, but the indexed event field records `msg.sender`. Off-chain indexers, explorers, and UIs tracking who received each reward will show incorrect addresses.

Recommended Mitigation:

```
emit Claimed(treasureHash, recipient);
```

[L-3] `nonReentrant` modifier applied to only one of three ETH-sending functions {#l-3-nonreentrant-incomplete-coverage}

Severity: Low

Description:

`claim()` is protected by `nonReentrant`, but `withdraw()` and `emergencyWithdraw()` are not. While those functions are currently protected by other conditions (`claimsCount >= MAX_TREASURES` and

`paused` respectively), reentrancy guards should be applied consistently to all ETH-transfer functions as a defense-in-depth measure.

Recommended Mitigation:

Add `nonReentrant` to `withdraw()` and `emergencyWithdraw()`.

[L-4] `recipient` public input unconstrained in ZK circuit — binding relies solely on proof system {#l-4-unconstrained-recipient}

Severity: Low **File:** `circuits/src/main.nr:28–39`

Description:

`recipient` is declared as a public input but is never referenced by any circuit constraint:

```
fn main(treasure: Field, treasure_hash: pub Field, recipient: pub Field) {
    assert(is_allowed(treasure_hash));
    assert(std::hash::pedersen_hash([treasure]) == treasure_hash);
    // recipient: zero constraints — not used anywhere
}
```

The circuit comment claims this binding "effectively mitigat[es] replay attacks," but it relies entirely on Barretenberg's proof system binding public inputs at the protocol level — not on any explicit circuit constraint. This is fragile: the guarantee is invisible to circuit auditors and would silently break if the backend changed or had a bug in public-input handling. Nargo itself emits a compiler warning confirming the input is unused:

```
warning: unused variable recipient
   ┌─ src/main.nr:28:52
28 │ fn main(treasure: Field, treasure_hash: pub Field, recipient: pub
Field) {
   │                                                    --------- unused
variable
```

Recommended Mitigation:

Add an explicit constraint tying `recipient` into the circuit, e.g. a no-op assertion that makes the dependency visible:

```
assert(recipient == recipient); // documents the binding intent explicitly
```

Or better, incorporate `recipient` into the hash computation so it is cryptographically entangled with the proof witness.

[L-5] ETH transfer to smart contract recipient may permanently brick a treasure claim {#l-5-push-payment-to-smart-contract}

Severity: Low

Description:

`claim()` pushes ETH directly to `recipient` using a low-level call:

```
// TreasureHunt.sol:107-108
(bool sent, ) = recipient.call{value: REWARD}("");
require(sent, "ETH_TRANSFER_FAILED");
```

If `recipient` is a smart contract that does not implement a `receive()` or `fallback()` function, or one that deliberately reverts on ETH receipt, the transfer fails and the entire `claim()` call reverts. Because `_markClaimed(treasureHash)` has already executed before the transfer (state was updated), the treasure would be marked claimed but no reward was paid — the treasure is permanently bricked.

Impact:

A participant who accidentally provides a smart contract address as recipient (e.g. a multisig without ETH receive support) loses their reward permanently. A malicious participant could also use this to grief the hunt by bricking treasures without collecting rewards.

Proof of Concept:

```
// Recipient contract that rejects ETH
contract NoReceive {
    // no receive() or fallback()
}

// claim() reverts at the ETH transfer — treasure bricked
hunt.claim(proof, treasureHash, payable(address(new NoReceive())));
```

Recommended Mitigation:

Use a pull-payment pattern — store the reward in a mapping and let the recipient withdraw it themselves:

```
mapping(address => uint256) public pendingRewards;

// in claim():
pendingRewards[recipient] += REWARD;
emit Claimed(treasureHash, recipient);

// new function:
function withdrawReward() external {
    uint256 amount = pendingRewards[msg.sender];
    if (amount == 0) revert NoRewardPending();
```

```
    pendingRewards[msg.sender] = 0;
    (bool sent, ) = msg.sender.call{value: amount}("");
    require(sent, "ETH_TRANSFER_FAILED");
}
```

This eliminates the push-payment risk entirely.

[L-6] `withdraw()` missing access control — any address can trigger owner withdrawal {#l-6-withdraw-missing-access-control}

Severity: Low

Description:

`withdraw()` has no `onlyOwner` check — any address can call it once all treasures are claimed:

```
// TreasureHunt.sol:223-232
function withdraw() external {
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");
    uint256 balance = address(this).balance;
    require(balance > 0, "NO_FUNDS_TO_WITHDRAW");
    (bool sent, ) = owner.call{value: balance}("");
    require(sent, "ETH_TRANSFER_FAILED");
    emit Withdrawn(balance, address(this).balance);
}
```

ETH still goes to `owner`, so this is not a theft vector. However, any random address can trigger the owner's withdrawal without their knowledge or consent — violating the principle that owner-only flows should be owner-controlled.

Impact:

Low — no funds are at risk. But a griefing actor could force the withdrawal at an inconvenient time, or front-run the owner's own `withdraw()` call.

Proof of Concept:

```
// randomUser triggers owner's withdrawal — should revert but doesn't
vm.prank(randomUser);
hunt.withdraw();
assertGt(OWNER.balance, ownerBefore, "ETH reached owner via unauthorized
trigger");
```

Recommended Mitigation:

Add the `onlyOwner` modifier or an inline check:

```
function withdraw() external {
    require(msg.sender == owner, "ONLY_OWNER");
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");
    ...
}
```

Informational

[I-1] 11 custom errors declared but never used {#i-1-unused-errors}

The following errors are defined but their corresponding `require()` calls still use string literals. This wastes gas on declaration and creates confusing inconsistency:

Error	File
`OwnerCannotBeRecipient`	TreasureHunt.sol:14
`HuntNotOver`	TreasureHunt.sol:16
`NoFundsToWithdraw`	TreasureHunt.sol:17
`OnlyOwnerCanFund`	TreasureHunt.sol:18
`OnlyOwnerCanPause`	TreasureHunt.sol:19
`OnlyOwnerCanUnpause`	TreasureHunt.sol:20
`TheContractMustBePaused`	TreasureHunt.sol:22
`OnlyOwnerCanUpdateVerifier`	TreasureHunt.sol:23
`OnlyOwnerCanEmergencyWithdraw`	TreasureHunt.sol:24
`InvalidAmount`	TreasureHunt.sol:25
`SumcheckFailed`	Verifier.sol:9

Replace all `require(condition, "STRING")` patterns with `if (!condition) revert CustomError()`, or remove unused declarations.

[I-2] Mixed error-handling styles (`require` strings vs. custom errors) {#i-2-mixed-error-handling}

Custom errors (e.g. `revert ContractPaused()`) are used in some places while `require(…, "STRING")` is used in others. Standardise on custom errors throughout — they are cheaper on gas and more ergonomic for tooling.

[I-3] Unspecific Solidity pragma {#i-3-unspecific-pragma}

Both files use `pragma solidity ^0.8.27`. Pin to an exact version (e.g. `pragma solidity 0.8.27`) for deterministic compilation across environments.

[I-4] PUSH0 opcode — EVM version compatibility {#i-4-push0-opcode}

Solidity ≥ 0.8.20 targets the Shanghai EVM by default, emitting `PUSH0` opcodes. Deployment will fail on L2 chains that do not yet support Shanghai. Explicitly set `evmVersion` in `foundry.toml` to match the target deployment chain.

[I-5] Unused state variable `_treasureHash` {#i-5-unused-state-variable}

`bytes32 private immutable _treasureHash` (line 35) is declared but never assigned or used for any legitimate purpose (see H-2). Remove it entirely after fixing the double-spend bug.